COMPUTER LABORATORY

LDA on the Iris Dataset

Complete Code Explanation • Theory • Viva & Cross-Questions

Practical focus
Linear Discriminant Analysis (LDA) for classifying Iris flowers into three species.

Based on your uploaded notebook
Assignment_1B.ipynb — pandas, train_test_split, LinearDiscriminantAnalysis, and classification metrics.

Level
Beginner-friendly explanations, with enough depth for practical viva questions.

A practical viva guide

Read the intuition first, then learn what each line does, and finally practise answering the questions aloud.

1. Understand the practical

Objective: Build a machine-learning classification model that predicts the species of an Iris flower from four measurements. Your notebook uses Linear Discriminant Analysis (LDA), a supervised learning algorithm.

What is the Iris dataset?

The Iris dataset is a small, commonly used dataset in machine learning. Each row represents one flower. It contains four numeric measurements and one label (the species). The model learns how the measurements differ across species.

Column	Meaning	Type
SepalLengthCm	Length of the sepal, in centimetres	Input feature
SepalWidthCm	Width of the sepal, in centimetres	Input feature
PetalLengthCm	Length of the petal, in centimetres	Input feature
PetalWidthCm	Width of the petal, in centimetres	Input feature
Species	Flower class/species to predict	Target label

The three target classes

- Iris-setosa, Iris-versicolor, and Iris-virginica.
- This is a multi-class classification problem because there are three possible output classes.
- It is supervised learning because the training examples already contain the correct species labels.

Input, output, and goal

Input (X): four measurements for a flower. Output (y): predicted species. Goal: learn a decision rule from labelled examples and use it on flowers not used during training.

Important distinction: the model does not identify a flower from an image. It classifies using the four numeric measurements supplied to it.

The assignment list describes LDA on the Iris dataset as classifying which species a given flower belongs to. The notebook implements that task.

2. LDA theory — explained simply

What is Linear Discriminant Analysis?

Linear Discriminant Analysis (LDA) is a supervised machine-learning method. It is used for classification and can also be used for dimensionality reduction. It tries to find combinations of input features that separate known classes well.

Intuition: separate groups, not just describe variation

Imagine plotting flowers using petal length and petal width. Setosa flowers tend to form a clearly separate group, while versicolor and virginica may be closer. LDA looks for a direction (or set of directions) where the class groups are well separated.

Two types of variation

- Within-class variation: how spread out the observations are inside the same species. LDA prefers this to be small along the discriminant direction.
- Between-class variation: how far apart the species groups are. LDA prefers this to be large.

In simple words: LDA tries to maximise separation between class centres while keeping each class internally compact.

How does LDA work?

1. It uses the known class labels in the training data. 2. It calculates the mean of each feature for each class. 3. It estimates how much data varies within each class and between classes. 4. It finds linear combinations of features that separate classes. 5. It uses those learned boundaries to classify a new observation.

Why is it called 'linear'?

The decision boundaries produced by standard LDA are linear (straight-line boundaries in two dimensions, or hyperplanes in higher dimensions). A new sample is assigned to the class with the strongest discriminant score/posterior probability under the model.

Important assumptions

- Observations within each class are approximately normally distributed (multivariate normality).
- Classes share a common covariance matrix (similar covariance structure).
- Observations are independent, and predictors should not be perfectly redundant/collinear.
- LDA can still be useful when assumptions are not perfectly met, but its performance or probability estimates may be affected.

Viva wording: “LDA is a supervised classification technique. It finds linear combinations of features that maximise between-class separation relative to within-class spread.”

3. Full notebook code

This is the code present in your uploaded notebook, arranged as one continuous program. The dataset file must be available at the path used by read_csv.

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris = pd.read_csv("iris.csv")

X = iris.drop(columns='Species')
y = iris['Species']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

display(iris.head())

# Model building
model = LinearDiscriminantAnalysis()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

flower = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]], columns=X.columns
)
print(model.predict(flower)[0])
```

Overall flow

Stage	What happens
1. Import	Load pandas and the required Scikit-learn tools.
2. Load data	Read iris.csv into a pandas DataFrame.
3. Separate X and y	X contains measurements; y contains species labels.
4. Split	Keep some labelled examples aside for testing.
5. Train	Fit LDA using the training measurements and labels.
6. Predict/evaluate	Predict test labels and compare with actual labels.
7. New flower	Predict the species for one manually supplied flower.

Remember: `fit()` learns from the training data; `predict()` uses what was learned. The test labels are not used to train the model.

4. Code explanation — imports, data, and splitting

1) import pandas as pd

Pandas is a Python library for handling tabular data. `pd` is a short alias used by convention.

- A DataFrame is like a spreadsheet: rows and columns.
- A Series is like one column of a DataFrame.
- The notebook uses pandas to read the CSV, select columns, and create a one-row input for prediction.

2) Scikit-learn imports

`train_test_split` divides data into training and testing parts. `LinearDiscriminantAnalysis` creates the LDA estimator. `accuracy_score`, `confusion_matrix`, and `classification_report` measure model performance.

3) iris = pd.read_csv('iris.csv')

Reads the CSV file and stores it in a DataFrame named `iris`. The file path must be correct relative to the notebook's current working directory. If the file is missing or the path is wrong, Python raises a file-not-found error.

4) X and y

```
X = iris.drop(columns='Species')
y = iris['Species']
```

`drop(columns='Species')` removes the target column from the input features. `iris['Species']` selects the target column. The capital letters X and y are common machine-learning notation: X = features, y = target.

Why remove Species from X? Because the model must predict the species. If the answer column were included among the inputs, the model would be given the answer during training — this is target leakage.

5) train_test_split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

- `test_size=0.2`: 20% of rows go to testing and 80% to training.
- `random_state=42`: fixes the random split so it is reproducible. 42 is not a special mathematical value.
- `stratify=y`: keeps approximately the same class proportions in train and test sets.
- The X and y splits must stay aligned: each row of X_train must correspond to the same row's label in y_train.

For a dataset with 150 rows, an 80/20 split gives approximately 120 training rows and 30 testing rows. Exact per-class counts depend on stratification and rounding.

5. Code explanation — model, prediction, evaluation

6) Create the model

```
model = LinearDiscriminantAnalysis()
```

This creates an LDA model object with Scikit-learn's default settings. At this point, the model has not learned from the Iris data yet.

7) Train the model with fit()

```
model.fit(X_train, y_train)
```

`fit()` learns the class-related statistics from the training data. It receives both features and correct labels because LDA is supervised. After fitting, the model stores the learned information needed for future predictions.

8) Predict with predict()

```
y_pred = model.predict(X_test)
```

`predict()` takes the unseen test features and returns one predicted class label for each row. `y_pred` contains predictions; `y_test` contains the true answers.

9) Evaluate

- accuracy_score(y_test, y_pred): fraction of test examples classified correctly. Accuracy = correct predictions / total predictions.
- confusion_matrix(y_test, y_pred): table comparing actual classes (rows) with predicted classes (columns) in Scikit-learn's default label order.
- classification_report(y_test, y_pred): prints precision, recall, F1-score, and support for each class, plus averages.

Metric	Meaning in plain language
Precision	Of all samples predicted as a class, how many truly belong to it?
Recall	Of all actual samples of a class, how many did the model find?
F1-score	Harmonic mean of precision and recall; balances both.
Support	Number of actual test examples belonging to that class.

10) Predict a manually entered flower

```
flower = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]], columns=X.columns
)
print(model.predict(flower)[0])
```

The nested list contains one sample with four measurements, in the same feature order used during training. `columns=X.columns` gives the DataFrame the expected feature names and order. `predict(flower)` returns an array of predicted labels; `[0]` extracts the first (and only) prediction.

Do not change the feature order or units. A model trained on centimetres and a particular column order expects new samples in the same format.

6. Important theory and comparison questions

1. What is the difference between classification and regression?

Answer: Classification predicts a category or class, such as a flower species. Regression predicts a continuous numeric value, such as a house price.

2. Is LDA supervised or unsupervised?

Answer: Supervised. It needs class labels during training to learn how the classes differ.

3. Can LDA be used for dimensionality reduction?

Answer: Yes. It projects data onto discriminant directions that preserve class separation. In this notebook, it is used as a classifier.

4. What is the maximum number of LDA components?

Answer: At most min(number of features, number of classes − 1). With 3 classes, at most 2 discriminant components can be formed.

5. What is the difference between PCA and LDA?

Answer: PCA is unsupervised and finds directions of maximum overall variance. LDA is supervised and finds directions that best separate labelled classes.

6. Does PCA use target labels?

Answer: No. Standard PCA only uses the feature matrix. LDA uses the class labels.

7. Which is better, PCA or LDA?

Answer: Neither is universally better. PCA is useful for compression and capturing overall variance; LDA is useful when class separation is the goal.

8. What is a covariance matrix?

Answer: It describes how features vary and how pairs of features vary together. Positive covariance means they tend to increase together; negative means one tends to increase as the other decreases.

9. What is a decision boundary?

Answer: A boundary separating regions assigned to different predicted classes. Standard LDA gives linear decision boundaries.

10. What does the word 'discriminant' mean here?

Answer: A function or direction that helps distinguish one class from another.

11. What happens if classes overlap heavily?

Answer: The model may confuse them and classification performance can decrease.

12. What are LDA's main assumptions?

Answer: Approximately multivariate-normal features within each class, a common covariance matrix across classes, independent observations, and no perfect multicollinearity.

13. Is feature scaling required for LDA?

Answer: Not always in the same way it is important for distance-based methods. LDA uses covariance estimates, so scale and numerical conditioning can still matter. Scaling should be considered based on the data and preprocessing pipeline.

14. Why is Iris a popular beginner dataset?

Answer: It is small, labelled, easy to understand, has numeric features, and provides a clear multi-class classification example.

15. What is multi-class classification?

Answer: A classification task with more than two possible classes. Iris has three species.

7. Code, functions, and tricky cross-questions

16. Why use `pd.read_csv()`?
Answer: To load comma-separated data into a pandas DataFrame. For Excel, pandas provides `read_excel()`.

17. What does `head()` do?
Answer: Displays the first five rows by default. It is a quick way to inspect whether the data loaded correctly.

18. What does `display()` do?
Answer: In Jupyter, it renders an object such as a DataFrame in a readable table format. `print()` displays a text representation.

19. Why use `drop(columns='Species')` instead of manually selecting four columns?
Answer: It removes the target column while keeping all remaining feature columns. This is concise, but one should still verify the remaining columns are appropriate.

20. What if the target column name is misspelled?
Answer: Pandas raises a KeyError because the requested column does not exist.

21. Why split before fitting the model?
Answer: To keep a test set separate and estimate performance on unseen examples. Fitting before splitting would leak test information into training.

22. What if `test_size=0.3`?
Answer: Approximately 30% is reserved for testing and 70% for training. The test set becomes larger and the training set smaller.

23. What if `random_state` is removed?
Answer: The split may change between runs, so the exact results can change.

24. Why use `stratify=y`?
Answer: It preserves class proportions in both subsets, which is useful for classification—especially if classes are imbalanced.

25. What if `stratify` is omitted?
Answer: The split is random and may produce different class proportions in train and test sets.

26. What is an estimator in Scikit-learn?
Answer: An object that learns patterns from data. Many estimators use `fit()` to learn and `predict()` to produce outputs.

27. What does `fit()` return?
Answer: For Scikit-learn estimators, it generally returns the fitted estimator itself, allowing method chaining.

28. What is the difference between `predict()` and `predict_proba()`?
Answer: `predict()` returns the chosen class label. `predict_proba()` returns estimated probabilities for each class, when supported.

29. What does `[0]` do in the last line?
Answer: It selects the first prediction from the returned array.

30. Why wrap the sample in a DataFrame?
Answer: It gives the sample named columns matching the training features, reducing the risk of feature-name/order mistakes.

31. What if we pass only three values instead of four?
Answer: The model will raise a feature-count/shape error because it was trained using four input features.

32. What if the sample columns are in a different order?
Answer: Predictions can be wrong or Scikit-learn may warn/error depending on how the input is supplied. Keep the same feature names and order.

33. What does `confusion_matrix` show?
Answer: Counts of actual-versus-predicted labels. The diagonal contains correct predictions; off-diagonal cells are mistakes.

34. Can accuracy alone be misleading?
Answer: Yes. With imbalanced classes, a model can have high accuracy while performing poorly on a minority class. Check per-class precision, recall, and F1.

8. More viva questions, limitations, and final revision

35. What is a false positive in multi-class classification?
Answer: A sample is predicted as a class even though its true class is another one. In a confusion matrix, it contributes to an off-diagonal cell for the predicted class.

36. What is a false negative for a class?
Answer: A sample truly belongs to that class but is predicted as a different class.

37. What is macro average?
Answer: The metric is calculated separately for each class and then averaged equally, regardless of class size.

38. What is weighted average?
Answer: The class metrics are averaged using each class's support as its weight.

39. What is support?
Answer: The number of true samples for a class in the evaluated dataset.

40. What does `stratify=y` require?
Answer: The target labels must have enough examples in each class to create the requested split. Very small classes can make stratified splitting fail.

41. What is overfitting?
Answer: When a model learns the training data too closely, including noise, and performs worse on new data.

42. Does this notebook use cross-validation?
Answer: No. It uses one train-test split. Cross-validation would evaluate the model over multiple splits.

43. Does this notebook handle missing values?
Answer: No explicit missing-value handling is shown. If the data contains missing values, preprocessing may be needed before fitting.

44. Does this notebook scale features?
Answer: No explicit scaling is shown.

45. How can you improve the evaluation?
Answer: Use a confusion matrix, per-class metrics, cross-validation, and a held-out test set. Also inspect data quality and class balance.

46. What is data leakage?
Answer: When information unavailable at prediction time accidentally enters training or evaluation, making performance look better than it really is.

47. What is the difference between training and testing data?
Answer: Training data is used to learn model parameters. Testing data is held aside to estimate performance on unseen examples.

48. What is a hyperparameter?
Answer: A setting chosen before or around training, rather than directly learned from the data. LDA has options such as solver and shrinkage.

49. Name a limitation of LDA.
Answer: Its assumptions may not fit all datasets; it can struggle when class distributions are strongly non-normal, covariance structures differ greatly, or classes overlap.

50. Why is the model output a species name rather than a number?
Answer: The target column contains categorical labels. The classifier predicts one of those labels.

Common mistakes to avoid

- Do not include `Species` inside X — it is the answer column.
- Do not train on X_test or use y_test during fitting.
- Do not change the order of the four flower measurements for a new prediction.
- Do not claim that PCA and LDA are the same: PCA ignores labels; LDA uses labels.
- Do not describe accuracy as the percentage of each class correctly predicted; it is the overall fraction correct.

30-second practical explanation

“This practical performs multi-class classification on the Iris dataset using Linear Discriminant Analysis. First, I load the CSV using pandas. I separate the four flower measurements into X and the Species column into y. Then I split the data into 80% training and 20% testing, using stratification to preserve class proportions. I create an LDA model, train it using fit(), and predict the species for the test set using predict(). I evaluate the predictions using accuracy, a confusion matrix, and a classification report. Finally, I create a one-row DataFrame for a new flower and predict its species.”

Quick glossary

Term	Remember it as
Feature	Input measurement used to make a prediction.
Target	Correct output the model learns to predict.
Fit	Learn from training data.
Predict	Produce an output for new inputs.
Accuracy	Overall fraction of correct predictions.
LDA	Supervised method that separates labelled classes.

Source basis: your uploaded Assignment_1B.ipynb and CL1_AssignmentList.pdf. The notebook's code is the basis for the code walkthrough; theory explanations add standard introductory machine-learning context.